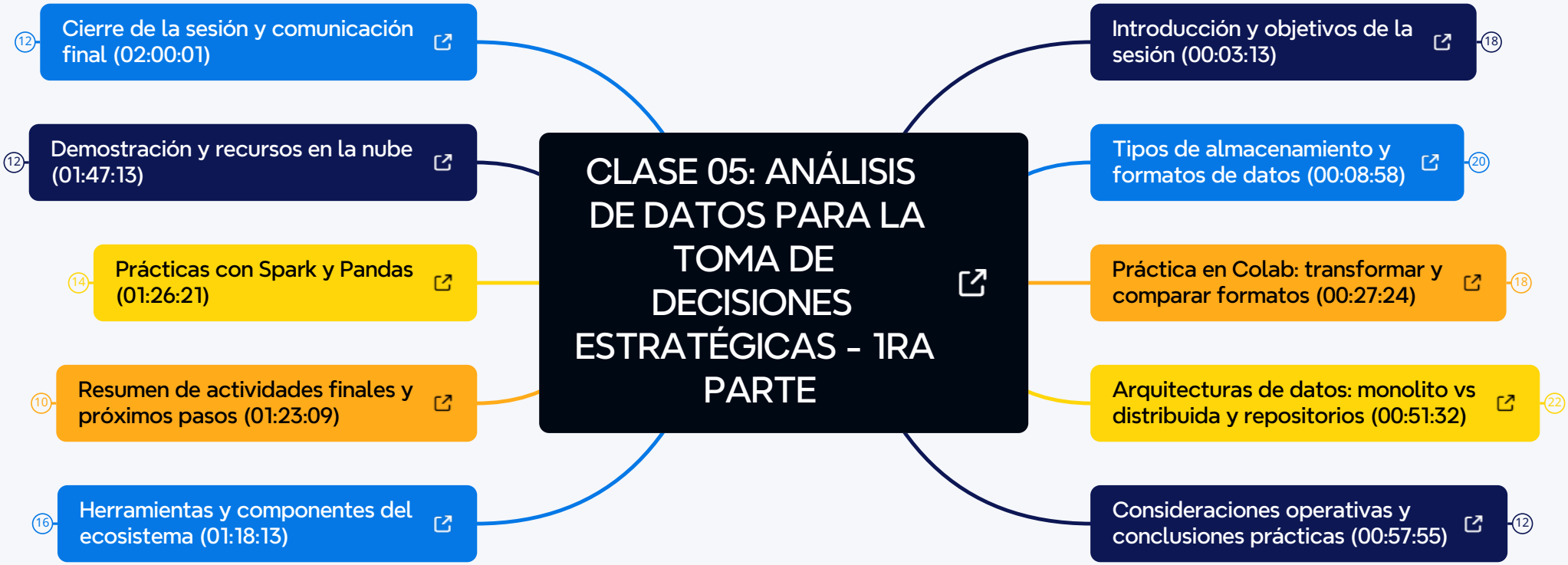
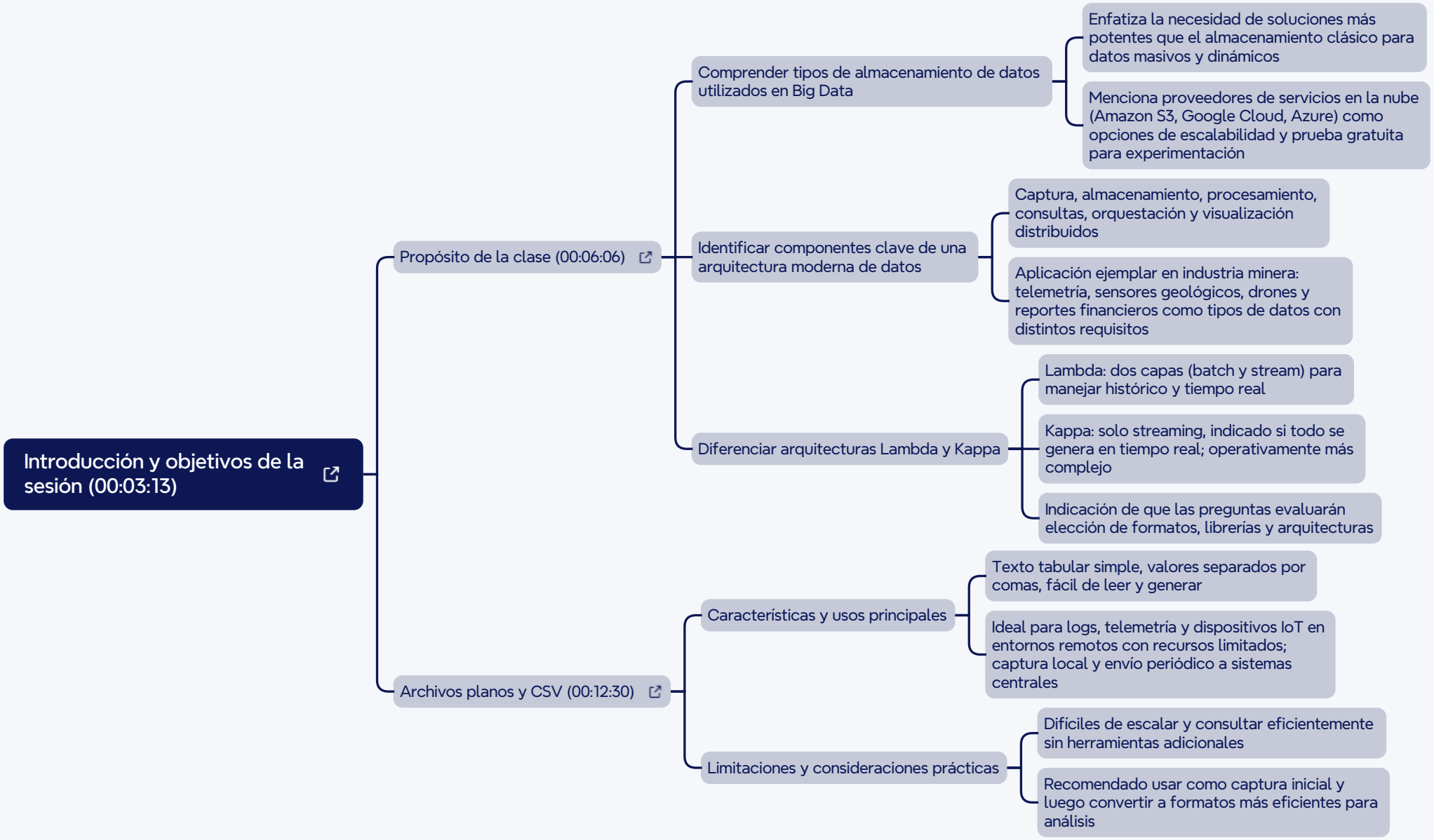
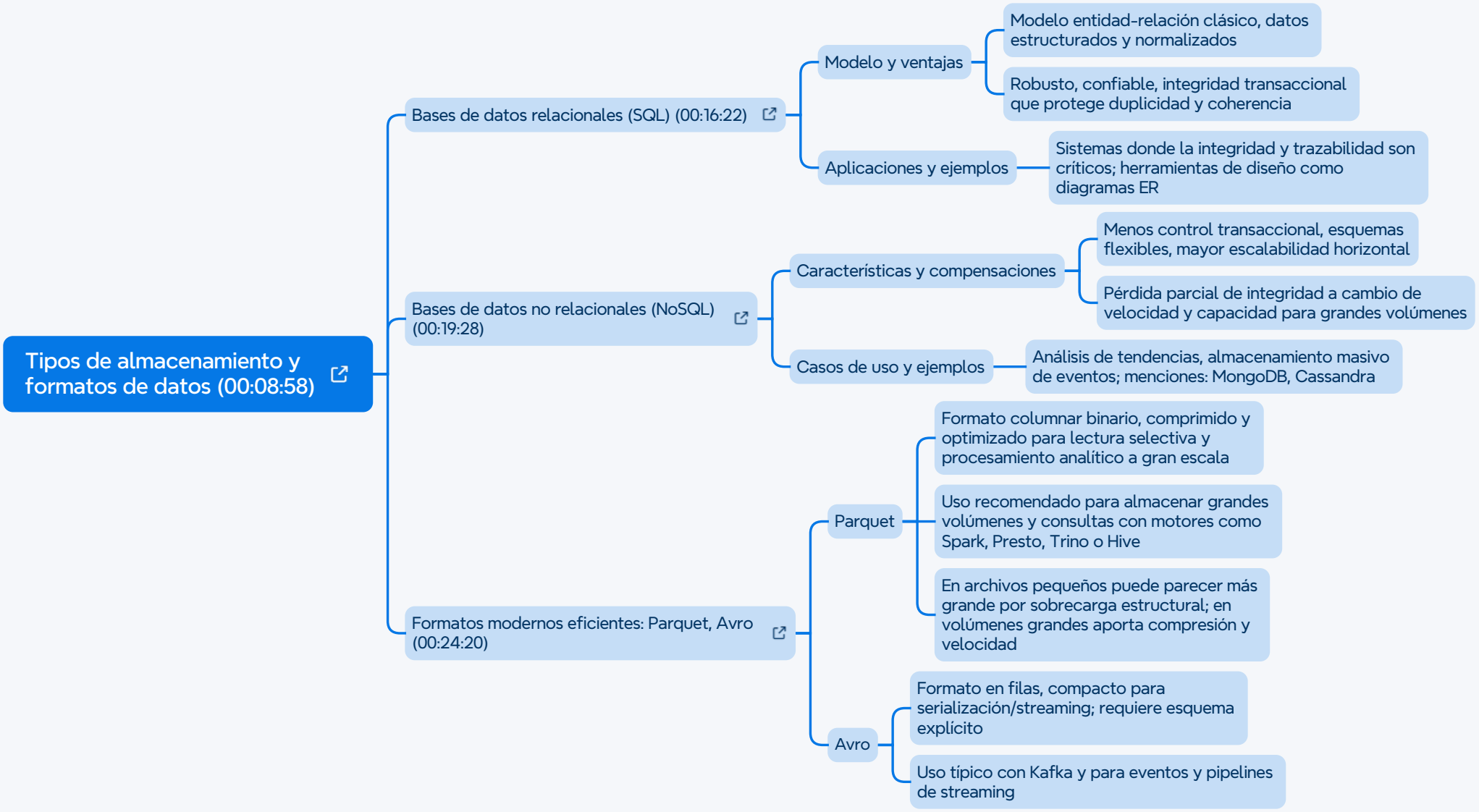
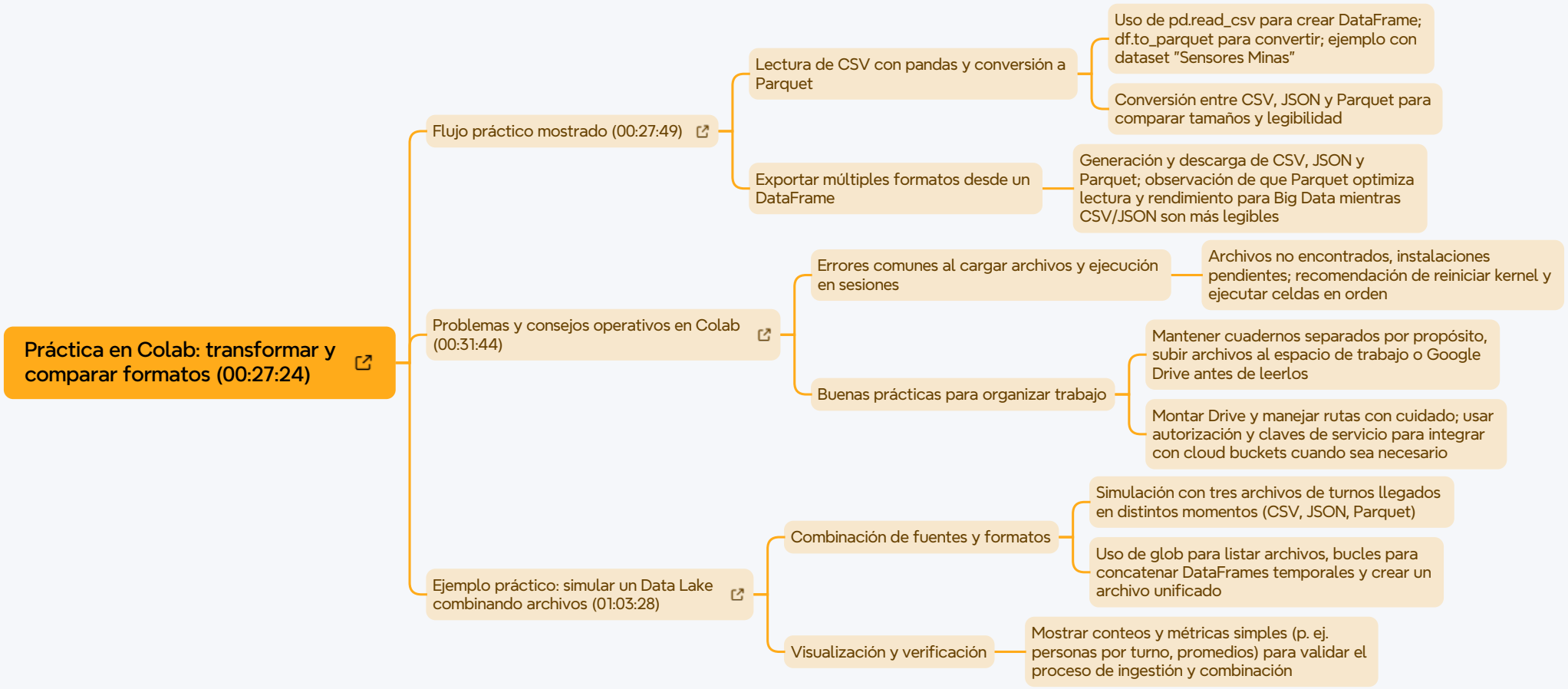
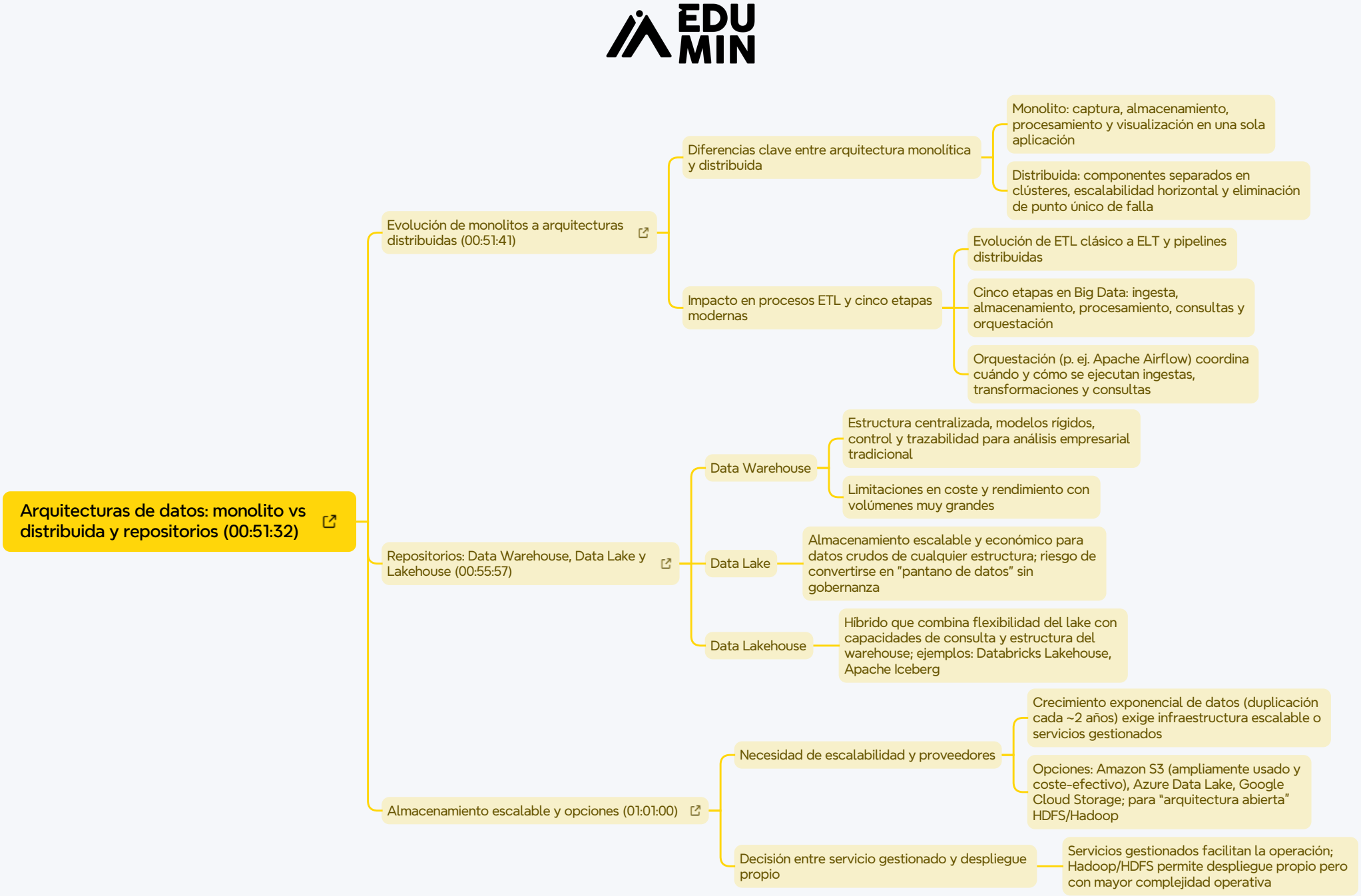
EDU MIN

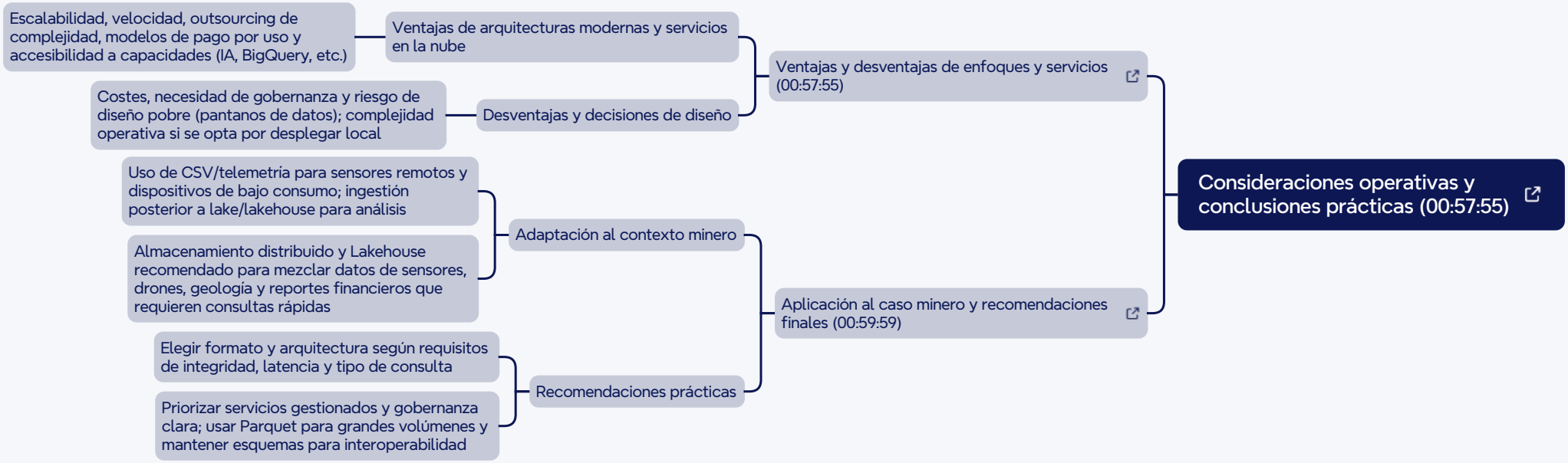

CLASE 05: ANÁLISIS DE DATOS PARA LA TOMA DE DECISIONES ESTRATÉGICAS - 1RA PARTE
Introducción y objetivos de la sesión (00:03:13)
18
Tipos de almacenamiento y formatos de datos (00:08:58)
20
Práctica en Colab: transformar y comparar formatos (00:27:24)
18
Arquitecturas de datos: monolito vs distribuida y repositorios (00:51:32)
22
Consideraciones operativas y conclusiones prácticas (00:57:55)
12
Cierre de la sesión y comunicación final (02:00:01)
12
Demostración y recursos en la nube (01:47:13)
12
Prácticas con Spark y Pandas (01:26:21)
14
Resumen de actividades finales y próximos pasos (01:23:09)
10
Herramientas y componentes del ecosistema (01:18:13)
16

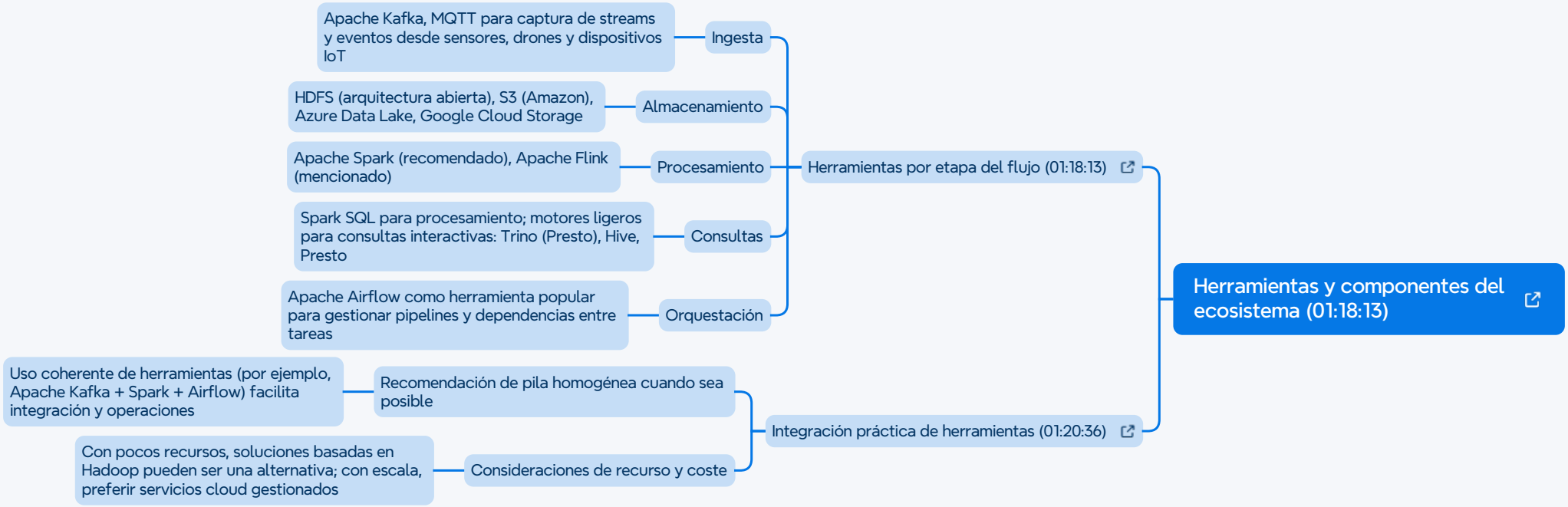

Introducción y objetivos de la sesión (00:03:13)
Propósito de la clase (00:06:06)
Comprender tipos de almacenamiento de datos utilizados en Big Data
Enfatiza la necesidad de soluciones más potentes que el almacenamiento clásico para datos masivos y dinámicos
Menciona proveedores de servicios en la nube (Amazon S3, Google Cloud, Azure) como opciones de escalabilidad y prueba gratuita para experimentación
Identificar componentes clave de una arquitectura moderna de datos
Captura, almacenamiento, procesamiento, consultas, orquestación y visualización distribuidos
Aplicación ejemplar en industria minera: telemetría, sensores geológicos, drones y reportes financieros como tipos de datos con distintos requisitos
Diferenciar arquitecturas Lambda y Kappa
Lambda: dos capas (batch y stream) para manejar histórico y tiempo real
Kappa: solo streaming, indicado si todo se genera en tiempo real; operativamente más complejo
Indicación de que las preguntas evaluarán elección de formatos, librerías y arquitecturas
Archivos planos y CSV (00:12:30)
Características y usos principales
Texto tabular simple, valores separados por comas, fácil de leer y generar
Ideal para logs, telemetría y dispositivos IoT en entornos remotos con recursos limitados; captura local y envío periódico a sistemas centrales
Limitaciones y consideraciones prácticas
Difíciles de escalar y consultar eficientemente sin herramientas adicionales
Recomendado usar como captura inicial y luego convertir a formatos más eficientes para análisis

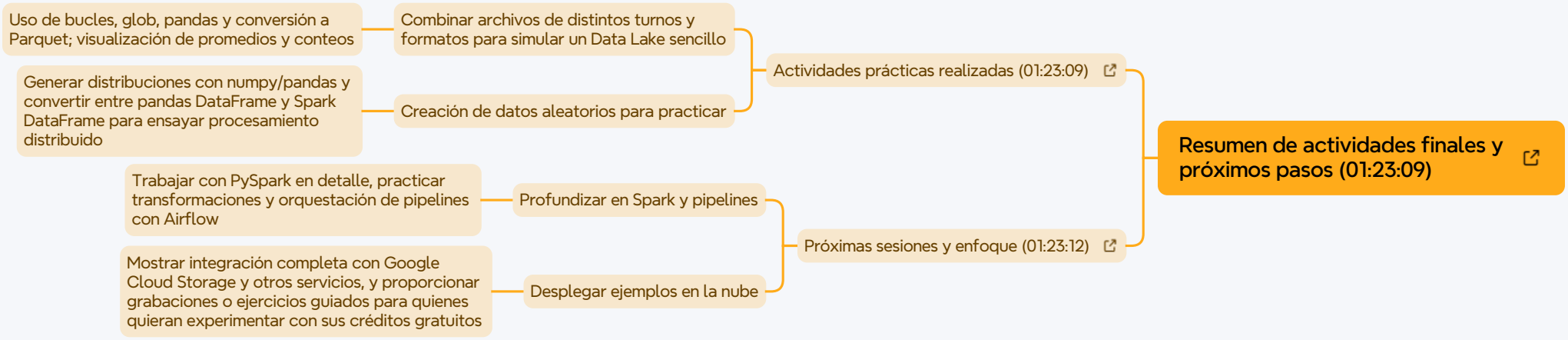

Tipos de almacenamiento y formatos de datos (00:08:58)
Bases de datos relacionales (SQL) (00:16:22)
Modelo y ventajas
Modelo entidad-relación clásico, datos estructurados y normalizados
Robusto, confiable, integridad transaccional que protege duplicidad y coherencia
Aplicaciones y ejemplos
Sistemas donde la integridad y trazabilidad son críticos; herramientas de diseño como diagramas ER
Bases de datos no relacionales (NoSQL) (00:19:28)
Características y compensaciones
Menos control transaccional, esquemas flexibles, mayor escalabilidad horizontal
Pérdida parcial de integridad a cambio de velocidad y capacidad para grandes volúmenes
Casos de uso y ejemplos
Análisis de tendencias, almacenamiento masivo de eventos; menciones: MongoDB, Cassandra
Formatos modernos eficientes: Parquet, Avro (00:24:20)
Parquet
Formato columnar binario, comprimido y optimizado para lectura selectiva y procesamiento analítico a gran escala
Uso recomendado para almacenar grandes volúmenes y consultas con motores como Spark, Presto, Trino o Hive
En archivos pequeños puede parecer más grande por sobrecarga estructural; en volúmenes grandes aporta compresión y velocidad
Avro
Formato en filas, compacto para serialización/streaming; requiere esquema explícito
Uso típico con Kafka y para eventos y pipelines de streaming

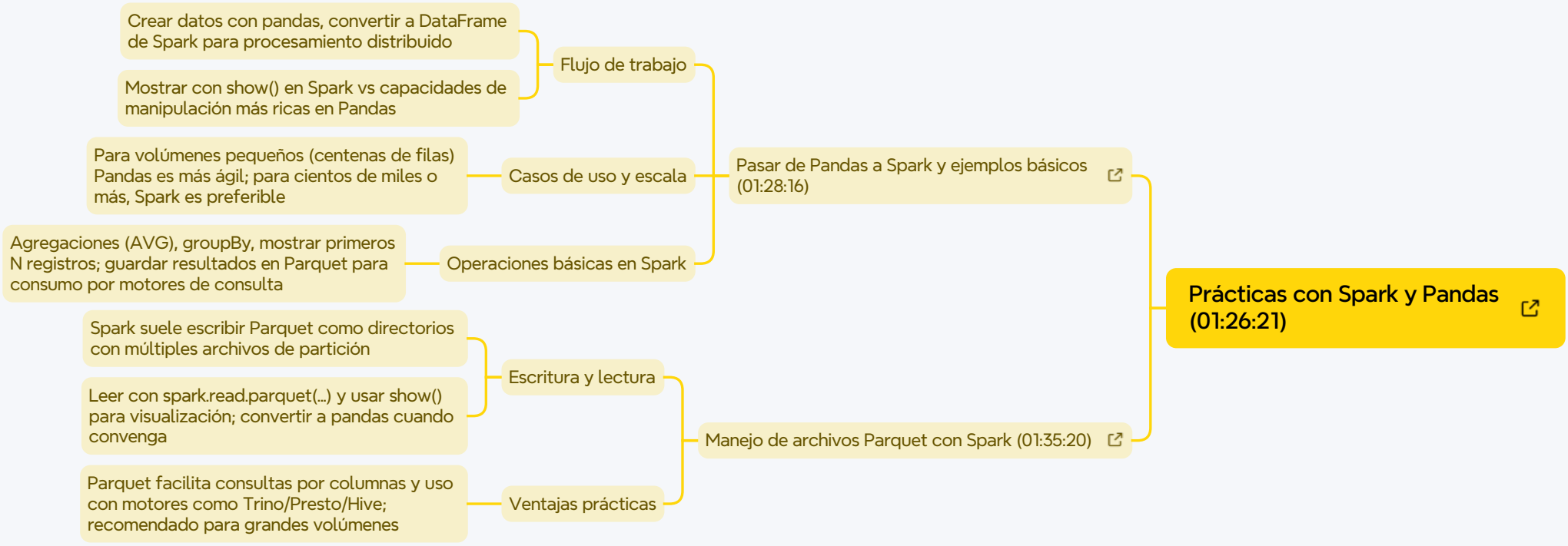

Práctica en Colab: transformar y comparar formatos (00:27:24)
Flujo práctico mostrado (00:27:49)
Lectura de CSV con pandas y conversión a Parquet
Uso de pd.read_csv para crear DataFrame; df.to_parquet para convertir; ejemplo con dataset "Sensores Minas"
Conversión entre CSV, JSON y Parquet para comparar tamaños y legibilidad
Exportar múltiples formatos desde un DataFrame
Generación y descarga de CSV, JSON y Parquet; observación de que Parquet optimiza lectura y rendimiento para Big Data mientras CSV/JSON son más legibles
Problemas y consejos operativos en Colab (00:31:44)
Errores comunes al cargar archivos y ejecución en sesiones
Archivos no encontrados, instalaciones pendientes; recomendación de reiniciar kernel y ejecutar celdas en orden
Buenas prácticas para organizar trabajo
Mantener cuadernos separados por propósito, subir archivos al espacio de trabajo o Google Drive antes de leerlos
Montar Drive y manejar rutas con cuidado; usar autorización y claves de servicio para integrar con cloud buckets cuando sea necesario
Ejemplo práctico: simular un Data Lake combinando archivos (01:03:28)
Combinación de fuentes y formatos
Simulación con tres archivos de turnos llegados en distintos momentos (CSV, JSON, Parquet)
Uso de glob para listar archivos, bucles para concatenar DataFrames temporales y crear un archivo unificado
Visualización y verificación
Mostrar conteos y métricas simples (p. ej. personas por turno, promedios) para validar el proceso de ingestión y combinación

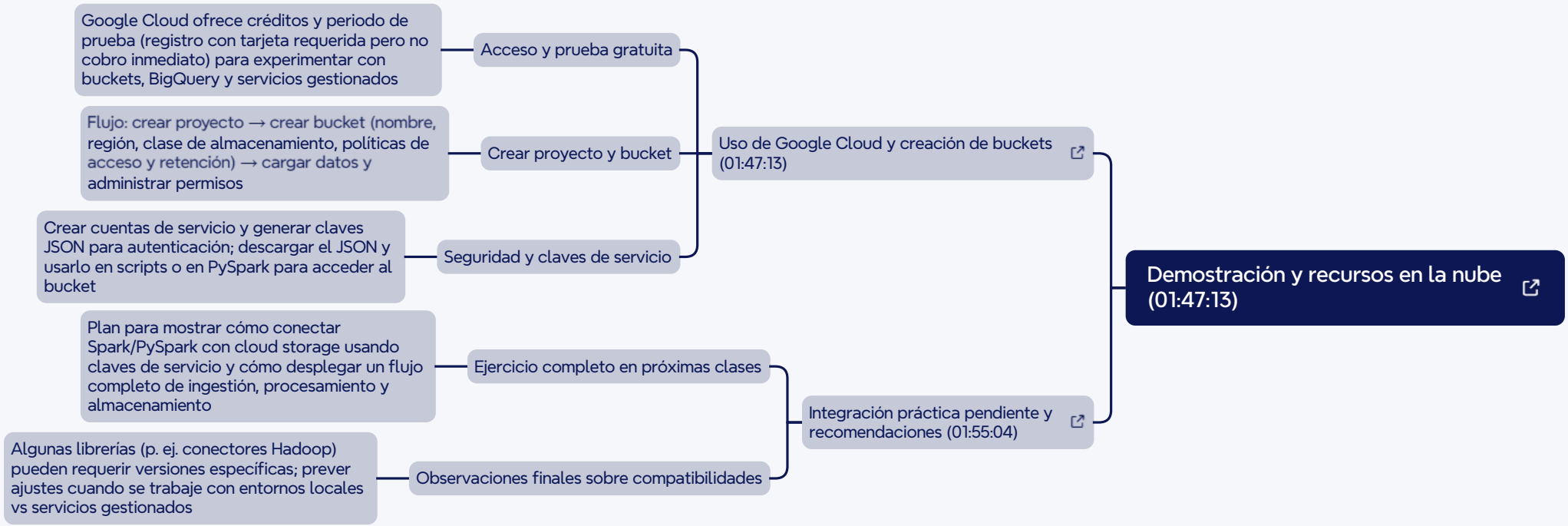

Arquitecturas de datos: monolito vs distribuida y repositorios (00:51:32)
Evolución de monolitos a arquitecturas distribuidas (00:51:41)
Diferencias clave entre arquitectura monolítica y distribuida
Monolito: captura, almacenamiento, procesamiento y visualización en una sola aplicación
Distribuida: componentes separados en clústeres, escalabilidad horizontal y eliminación de punto único de falla
Impacto en procesos ETL y cinco etapas modernas
Evolución de ETL clásico a ELT y pipelines distribuidas
Cinco etapas en Big Data: ingesta, almacenamiento, procesamiento, consultas y orquestación
Orquestación (p. ej. Apache Airflow) coordina cuándo y cómo se ejecutan ingestas, transformaciones y consultas
Repositorios: Data Warehouse, Data Lake y Lakehouse (00:55:57)
Data Warehouse
Estructura centralizada, modelos rígidos, control y trazabilidad para análisis empresarial tradicional
Limitaciones en coste y rendimiento con volúmenes muy grandes
Data Lake
Almacenamiento escalable y económico para datos crudos de cualquier estructura; riesgo de convertirse en "pantano de datos" sin gobernanza
Data Lakehouse
Híbrido que combina flexibilidad del lake con capacidades de consulta y estructura del warehouse; ejemplos: Databricks Lakehouse, Apache Iceberg
Almacenamiento escalable y opciones (01:01:00)
Necesidad de escalabilidad y proveedores
Crecimiento exponencial de datos (duplicación cada ~2 años) exige infraestructura escalable o servicios gestionados
Opciones: Amazon S3 (ampliamente usado y coste-efectivo), Azure Data Lake, Google Cloud Storage; para "arquitectura abierta" HDFS/Hadoop
Decisión entre servicio gestionado y despliegue propio
Servicios gestionados facilitan la operación; Hadoop/HDFS permite despliegue propio pero con mayor complejidad operativa

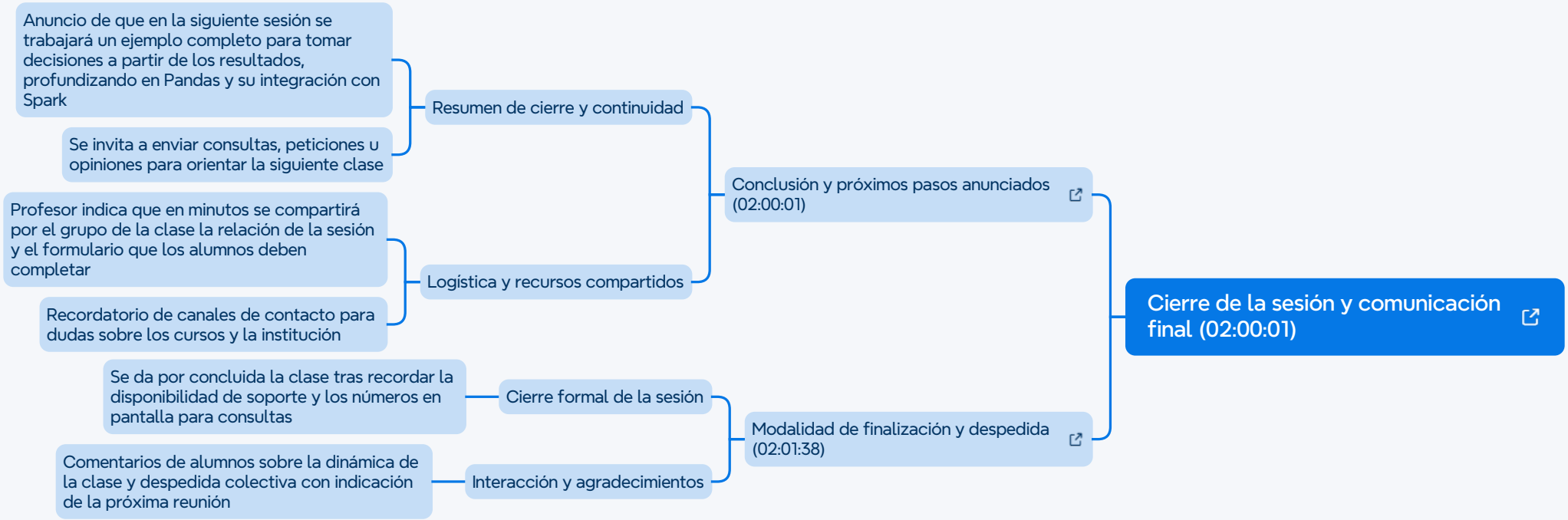

Consideraciones operativas y conclusiones prácticas (00:57:55)
Ventajas y desventajas de enfoques y servicios (00:57:55)
Ventajas de arquitecturas modernas y servicios en la nube
Escalabilidad, velocidad, outsourcing de complejidad, modelos de pago por uso y accesibilidad a capacidades (IA, BigQuery, etc.)
Desventajas y decisiones de diseño
Costes, necesidad de gobernanza y riesgo de diseño pobre (pantanos de datos); complejidad operativa si se opta por desplegar local
Aplicación al caso minero y recomendaciones finales (00:59:59)
Adaptación al contexto minero
Uso de CSV/telemetría para sensores remotos y dispositivos de bajo consumo; ingestión posterior a lake/lakehouse para análisis
Almacenamiento distribuido y Lakehouse recomendado para mezclar datos de sensores, drones, geología y reportes financieros que requieren consultas rápidas
Recomendaciones prácticas
Elegir formato y arquitectura según requisitos de integridad, latencia y tipo de consulta
Priorizar servicios gestionados y gobernanza clara; usar Parquet para grandes volúmenes y mantener esquemas para interoperabilidad


Herramientas y componentes del ecosistema (01:18:13)
Herramientas por etapa del flujo (01:18:13)
Ingesta
Apache Kafka, MQTT para captura de streams y eventos desde sensores, drones y dispositivos IoT
Almacenamiento
HDFS (arquitectura abierta), S3 (Amazon), Azure Data Lake, Google Cloud Storage
Procesamiento
Apache Spark (recomendado), Apache Flink (mencionado)
Consultas
Spark SQL para procesamiento; motores ligeros para consultas interactivas: Trino (Presto), Hive, Presto
Orquestación
Apache Airflow como herramienta popular para gestionar pipelines y dependencias entre tareas
Integración práctica de herramientas (01:20:36)
Recomendación de pila homogénea cuando sea posible
Uso coherente de herramientas (por ejemplo, Apache Kafka + Spark + Airflow) facilita integración y operaciones
Consideraciones de recurso y coste
Con pocos recursos, soluciones basadas en Hadoop pueden ser una alternativa; con escala, preferir servicios cloud gestionados


Resumen de actividades finales y próximos pasos (01:23:09)
Actividades prácticas realizadas (01:23:09)
Combinar archivos de distintos turnos y formatos para simular un Data Lake sencillo
Uso de bucles, glob, pandas y conversión a Parquet; visualización de promedios y conteos
Creación de datos aleatorios para practicar
Generar distribuciones con numpy/pandas y convertir entre pandas DataFrame y Spark DataFrame para ensayar procesamiento distribuido
Próximas sesiones y enfoque (01:23:12)
Profundizar en Spark y pipelines
Trabajar con PySpark en detalle, practicar transformaciones y orquestación de pipelines con Airflow
Desplegar ejemplos en la nube
Mostrar integración completa con Google Cloud Storage y otros servicios, y proporcionar grabaciones o ejercicios guiados para quienes quieran experimentar con sus créditos gratuitos


Prácticas con Spark y Pandas (01:26:21)
Pasar de Pandas a Spark y ejemplos básicos (01:28:16)
Flujo de trabajo
Crear datos con pandas, convertir a DataFrame de Spark para procesamiento distribuido
Mostrar con show() en Spark vs capacidades de manipulación más ricas en Pandas
Casos de uso y escala
Para volúmenes pequeños (centenas de filas) Pandas es más ágil; para cientos de miles o más, Spark es preferible
Operaciones básicas en Spark
Agregaciones (AVG), groupBy, mostrar primeros N registros; guardar resultados en Parquet para consumo por motores de consulta
Manejo de archivos Parquet con Spark (01:35:20)
Escritura y lectura
Spark suele escribir Parquet como directorios con múltiples archivos de partición
Leer con spark.read.parquet(...) y usar show() para visualización; convertir a pandas cuando convenga
Ventajas prácticas
Parquet facilita consultas por columnas y uso con motores como Trino/Presto/Hive; recomendado para grandes volúmenes


Demostración y recursos en la nube (01:47:13)
Uso de Google Cloud y creación de buckets (01:47:13)
Acceso y prueba gratuita
Google Cloud ofrece créditos y periodo de prueba (registro con tarjeta requerida pero no cobro inmediato) para experimentar con buckets, BigQuery y servicios gestionados
Crear proyecto y bucket
Flujo: crear proyecto → crear bucket (nombre, región, clase de almacenamiento, políticas de acceso y retención) → cargar datos y administrar permisos
Seguridad y claves de servicio
Crear cuentas de servicio y generar claves JSON para autenticación; descargar el JSON y usarlo en scripts o en PySpark para acceder al bucket
Integración práctica pendiente y recomendaciones (01:55:04)
Ejercicio completo en próximas clases
Plan para mostrar cómo conectar Spark/PySpark con cloud storage usando claves de servicio y cómo desplegar un flujo completo de ingestión, procesamiento y almacenamiento
Observaciones finales sobre compatibilidades
Algunas librerías (p. ej. conectores Hadoop) pueden requerir versiones específicas; prever ajustes cuando se trabaje con entornos locales vs servicios gestionados


Cierre de la sesión y comunicación final (02:00:01)
Conclusión y próximos pasos anunciados (02:00:01)
Modalidad de finalización y despedida (02:01:38)
Resumen de cierre y continuidad
Logística y recursos compartidos
Cierre formal de la sesión
Interacción y agradecimientos
Anuncio de que en la siguiente sesión se trabajará un ejemplo completo para tomar decisiones a partir de los resultados, profundizando en Pandas y su integración con Spark
Se invita a enviar consultas, peticiones u opiniones para orientar la siguiente clase
Profesor indica que en minutos se compartirá por el grupo de la clase la relación de la sesión y el formulario que los alumnos deben completar
Recordatorio de canales de contacto para dudas sobre los cursos y la institución
Se da por concluida la clase tras recordar la disponibilidad de soporte y los números en pantalla para consultas
Comentarios de alumnos sobre la dinámica de la clase y despedida colectiva con indicación de la próxima reunión